RestClient in Spring Boot

Learning Objectives

After completing this chapter, you will be able to:

- Understand why one microservice needs to communicate with another.
- Learn what RestClient is.
- Understand how RestClient works internally.
- Configure RestClient in Spring Boot.
- Build a complete communication between Customer Service and Order Service.
- Handle errors returned by another microservice.
- Follow best practices while using RestClient.

Introduction

In a Microservices Architecture, every service is independent.

For example:

- Customer Service manages customers.
- Order Service manages orders.
- Product Service manages products.
- Payment Service manages payments.

Each service owns its own database.

Suppose a customer places an order.

Before creating the order, Order Service must answer a simple question:

> **Does this customer really exist?**

Why Can't Order Service Access Customer Database?

Many beginners ask this question.

Instead of calling Customer Service, why not simply connect to the Customer database?

The answer is **No**.

Every microservice owns its database.

Consider this architecture.

What is RestClient?

`RestClient` is Spring Framework's modern HTTP client introduced in Spring Framework 6.

It replaces the older `RestTemplate`.

RestClient allows one application to send HTTP requests to another application.

Think of it as a Java object that knows how to communicate with REST APIs.

Example:

```
Order Service

↓

RestClient

↓
```

Real-World Analogy

Imagine you are ordering food.

You don't enter the restaurant's kitchen.

Instead:

- You speak to the waiter.
- The waiter talks to the chef.
- The chef prepares the food.
- The waiter brings the food back.

```
Customer

↓

Waiter
```

How RestClient Works Internally

Suppose Order Service wants customer information.

The sequence looks like this.

```
User
↓
POST /orders
↓
OrderController
↓
OrderService
↓
RestClient
↓
HTTP GET
↓
Customer Service
↓
CustomerController
↓
CustomerService
↓
Customer Database
↓
Customer Object
↓
JSON Response
↓
```

Project Structure

We have two Spring Boot applications.

```
customer-service

src
  ├── controller
  ├── service
  ├── repository
  ├── entity
  ├── dto
  └── resources
```

```
order-service

src
  ├── controller
  ├── service
  ├── repository
```

Maven Dependency

If you are using Spring Boot 3.x, you only need the Web starter.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

No additional RestClient dependency is required because it is part of Spring Framework 6.

Customer Entity

```
public class Customer {

    private Long id;

    private String name;

    private String email;

    private String phone;

}
```

Explanation

This class represents customer data received from Customer Service.

Order Service only needs these fields.

Customer Controller

```
@RestController
@RequestMapping("/customers")
public class CustomerController {

    @GetMapping("/{id}")
    public Customer getCustomer(@PathVariable Long id){

        return new Customer(
                id,
                "John",
                "john@gmail.com",
                "9876543210"
        );

    }

}
```

Explanation

`@RestController`

Marks the class as a REST Controller.

Spring automatically converts Java objects into JSON.

```
@RequestMapping("/customers")
```

Base URL.

Every endpoint starts with `/customers`.

`@GetMapping("/{id}")`

Handles HTTP GET requests.

Example

```
GET /customers/101
```

`@PathVariable`

Reads the value from the URL.

```
GET /customers/101
```

`id = 101`

JSON Response

When the API executes,

Customer Service returns

```
{
  "id":101,
  "name":"John",
  "email":"john@gmail.com",
  "phone":"9876543210"
}
```

Order Service receives this JSON.

Configuring RestClient

Create a configuration class.

```
config

└── RestClientConfig.java
```

```java
@Configuration
public class RestClientConfig {

    @Bean
    public RestClient restClient(){

        return RestClient.builder().build();

    }

}
```

Explanation

`@Configuration`

Tells Spring this class contains configuration.

`@Bean`

Creates a singleton RestClient object.

Spring stores it inside the IOC Container.

Whenever RestClient is required,

Spring injects this object.

Injecting RestClient

```
@Service
public class OrderService {

    private final RestClient restClient;

    public OrderService(RestClient restClient){

        this.restClient = restClient;

    }

}
```

Why Constructor Injection?

Constructor Injection is recommended because

- dependencies are mandatory
- objects become immutable
- easier to test
- preferred by Spring Boot

Avoid

```
@Autowired
private RestClient restClient;
```

Constructor injection is cleaner and follows modern Spring practices.

Calling Customer Service

```
Customer customer = restClient.get()
        .uri("http://localhost:8081/customers/{id}", customerId)
        .retrieve()
        .body(Customer.class);
```

This single statement performs the complete HTTP request.

Understanding Every Line

Step 1

```
restClient.get()
```

Create a GET request.

Step 2

```
.uri("http://localhost:8081/customers/{id}", customerId)
```

Builds the URL.

If

```
customerId = 101
```

The final URL becomes

```
http://localhost:8081/customers/101
```

Step 3

```
.retrieve()
```

Sends the HTTP request.

The application now waits for Customer Service.

Step 4

```
.body(Customer.class)
```

Reads the JSON response.

Spring automatically converts

```
{
 "id":101,
 "name":"John"
}
```

into

```
Customer customer
```

This process is called **JSON Deserialization**.

Complete Flow

```
POST /orders
↓
OrderController
↓
OrderService
↓
RestClient
↓
GET /customers/101
↓
Customer Service
↓
Customer Object
↓
JSON
↓
RestClient
↓
Customer Object
↓
Save Order
↓
Return Success
```

What Happens if Customer Doesn't Exist?

Suppose

```
GET /customers/999
```

returns

```
404 NOT FOUND
```

Then RestClient throws an exception.

Example:

```
try {

    Customer customer = restClient.get()
            .uri(url)
            .retrieve()
```

Best Practices

✓ Use Constructor Injection.

✓ Keep URLs in configuration files instead of hardcoding them.

✓ Validate all responses before processing.

✓ Handle HTTP errors gracefully.

✓ Use DTOs instead of exposing entities directly.

✓ Log every external API call for debugging.

✓ Configure connection and read timeouts.

✓ Use service discovery (Eureka or Kubernetes DNS) instead of hardcoded `localhost` URLs in production.

Common Mistakes

Directly accessing another service's database.

Hardcoding URLs throughout the codebase.

Ignoring HTTP status codes like `404` or `500`.

Not handling exceptions from external services.

Returning database entities directly to clients instead of using DTOs.

Interview Questions

1. What is RestClient?

RestClient is the modern synchronous HTTP client introduced in Spring Framework 6 for communicating with REST APIs.

2. Why is RestClient preferred over RestTemplate?

- Modern API design.
- Fluent and readable syntax.
- Official replacement for new Spring applications.
- Better integration with current Spring features.

3. What format does RestClient typically send and receive?

JSON over HTTP.

4. Can RestClient automatically convert JSON to Java objects?

Yes. Spring uses the configured `HttpMessageConverters` (typically backed by Jackson) to deserialize JSON into Java objects.

5. What happens if the target service returns HTTP 404?

`retrieve()` throws an exception unless you configure custom status handling.

Summary

In this chapter, you learned:

- Why microservices communicate using REST APIs.
- Why each service owns its own database.
- What RestClient is and why it replaced RestTemplate.
- How RestClient works internally.
- How to configure and inject RestClient in Spring Boot.
- How to call another microservice using RestClient.
- How JSON responses are converted into Java objects.
- Common errors, best practices, and interview questions.

re practical and closer to how enterprise microservice projects are built.